Documentación de pruebas unitarias PasswordChecker



Realizado por Robert Mosquera Rocamora.

Repositorio: [unit_test_rmr]

ÍNDICE

1. Estructura de los tests 3
2. Clases de equivalencia 3
3. Descripción detallada por grupo 4
   3.1. Prueba básica de contraseña correcta 4
   3.2. Longitud mínima y máxima 5
   3.3. Reglas de dígitos 6
   3.4. Reglas de mayúsculas y minúsculas 7
   3.5. Coincidencia entre contraseñas 8
   3.6. Contraseña en blanco 9
   3.7. Validación de la configuración 9
   3.8. Getters y valores iniciales 10
4. Test que se espera que falle y posible corrección 11
6. Conclusión 11

1. Estructura de los tests

La clase utiliza JUnit 5. El método setUp(), anotado con @BeforeEach, crea una instancia nueva de PasswordChecker antes de cada prueba, evitando que la configuración de un test afecte a otro. Los casos repetitivos se implementan mediante @ParameterizedTest y @ValueSource.

```java
@BeforeEach  robmosroc
void setUp() { checker = new PasswordChecker(); }

//Test de prueba
@Test  robmosroc
void passwordCorrecto() {
    int resultado = checker.check( password: "abcde123",  repeatPassword: "abcde123");
    assertEquals(PasswordChecker.PASSWORD_OK, resultado);
}
```

```java
@ParameterizedTest  robmosroc
@ValueSource(ints = { 1,5,7})
void longitudMinimaIncorrecta(int longitud) {
    checker.setMinLength(8);

    String password = "A".repeat(longitud);

    assertEquals(
            PasswordChecker.ERR_LENGTH,
            checker.check(password, password)
    );
}
```

Las comprobaciones usan assertEquals para comparar el código devuelto con el resultado esperado, assertThrows para verificar excepciones y assertFalse para comprobar que determinadas opciones están desactivadas inicialmente.

2. Clases de equivalencia

Una clase de equivalencia agrupa entradas que deberían producir el mismo comportamiento. Los valores representativos elegidos permiten comprobar tanto entradas aceptadas como entradas que deben ser rechazadas.

Clase / escenario	Entradas representativas	Resultado esperado
Longitud inferior al mínimo	Contraseñas con longitud 1, 5 y 7; mínimo configurado en 8.	ERR_LENGTH
Longitud superior al máximo	Contraseñas con longitud 11 y 23; máximo configurado en 10.	ERR_LENGTH
Longitud dentro del intervalo mínimo	Longitudes 8, 9 y 10 con mínimo configurado en 8.	PASSWORD_OK
Longitud dentro del intervalo máximo	Longitudes 8, 9 y 10 con máximo configurado en 10.	PASSWORD_OK
Sin dígitos, comprobación activada	Abcdefgh, Password, HolaMundo; checkDigits=true.	ERR_DIGITS
Comprobación de dígitos desactivada	Abcdefgh, Password, HolaMundo1; checkDigits=false.	PASSWORD_OK
Sin mayúsculas, comprobación activada	abcdefgh, password, holamundo; checkUppers=true.	ERR_UPPERS
Sin minúsculas, comprobación activada	ABCDEFGH, PASSWORD, HOLAMUNDO; checkLowers=true.	ERR_LOWERS
Sin mayúsculas, comprobación desactivada	abcdefgh, password, Holamundo; checkUppers=false.	PASSWORD_OK
Sin minúsculas, comprobación desactivada	ABCDEFGH, PAssword, holamundo; checkLowers=false.	PASSWORD_OK
Contraseñas coincidentes	abcdefgh, password y HOLAMUNDO, repetidas en ambos argumentos.	PASSWORD_OK
Contraseñas no coincidentes	El segundo argumento añade “hola” al primero.	ERR_NO_MATCH
Contraseña en blanco	Cadena vacía, un espacio y tres espacios.	ERR_BLANK
Configuración de longitud mínima inválida	Intento de establecer minLength=-1.	IllegalArgumentException
Configuración de longitud máxima inválida	minLength=10 y posterior intento de establecer maxLength=9.	IllegalArgumentException
Valores iniciales de longitud	getMinLength() y getMaxLength() al crear el comprobador.	8 e Integer.MAX_VALUE
Valores iniciales de comprobaciones	getCheckDigits(), getCheckUppers() y getCheckLowers().	false, false y false

3. Descripción detallada por grupo

3.1. Prueba básica de contraseña correcta

passwordCorrecto() comprueba que “abcde123”, introducida dos veces, se acepta y devuelve PASSWORD_OK. Es una prueba de humo que verifica un caso de funcionamiento general.

```java
@Test  robmosroc
void passwordCorrecto() {
    int resultado = checker.check( password: "abcde123",  repeatPassword: "abcde123");
    assertEquals(PasswordChecker.PASSWORD_OK, resultado);
}
```

3.2. Longitud mínima y máxima

longitudMinimaIncorrecta() verifica que las longitudes por debajo de 8 se rechazan. longitudMinimaCorrecta() comprueba longitudes 8, 9 y 10 con mínimo 8. longitudMaximaIncorrecta() verifica que las longitudes superiores a 10 se rechazan. longitudMaximaCorrecta() comprueba longitudes 8, 9 y 10 con máximo 10. Los tests usan cadenas compuestas por la letra A repetida la cantidad indicada de veces.

```
//comprobar que una longitud menor de la indicada en minlength da error
@ParameterizedTest  robmosroc
@ValueSource(ints = { 1,5,7})
void longitudMinimaIncorrecta(int longitud) {
    checker.setMinLength(8);

    String password = "A".repeat(longitud);

    assertEquals(
            PasswordChecker.ERR_LENGTH,
            checker.check(password, password)
    );
}

//comprobar que una longitud mayor de la indicada en maxlength da error
//El test está bien diseñado pero da error. Habría que comunicarselo a desarrollo
@ParameterizedTest  robmosroc
@ValueSource(ints = { 11,23})
void longitudMaximaIncorrecta(int longitud) {
    checker.setMaxLength(10);

    String password = "A".repeat(longitud);

    assertEquals(
            PasswordChecker.ERR_LENGTH,
            checker.check(password, password)
    );
}
```

3.3. Reglas de dígitos

faltaDigito() activa la obligación de incluir dígitos y comprueba que tres contraseñas sin números devuelven ERR_DIGITS. checkDigitosDesactivado() desactiva esa regla y verifica que las contraseñas proporcionadas se aceptan aunque no incluyan dígitos.

```java
// Comprobar que una contraseña sin dígitos da error
@ParameterizedTest  robmosroc
@ValueSource(strings = {"Abcdefgh", "Password", "HolaMundo"})
void faltaDigito(String password) {
    checker.setCheckDigits(true);

    assertEquals(
            PasswordChecker.ERR_DIGITS,
            checker.check(password, password)
    );
}

// Comprobar que se acepta una contraseña con y sin dígitos cuando el check de dígitos está desactivado
@ParameterizedTest  robmosroc
@ValueSource(strings = {"Abcdefgh", "Password", "HolaMundo1"})
void checkDigitosDesactivado(String password) {
    checker.setCheckDigits(false);

    assertEquals(
            PasswordChecker.PASSWORD_OK,
            checker.check(password, password)
    );
}
```

3.4. Reglas de mayúsculas y minúsculas

faltaMayusculas() activa la regla de mayúsculas y espera ERR_UPPERS cuando la contraseña no contiene ninguna. faltaMinusculas() activa la regla de minúsculas y espera ERR_LOWERS cuando no contiene ninguna. Los métodos checkMayusculasDesactivado() y checkMinusculasDesactivado() verifican que desactivar la regla correspondiente permite aceptar contraseñas que carecen de ese tipo de letra.

```
// Comprobar que una contraseña sin mayúsculas da error
@ParameterizedTest  robmosroc
@ValueSource(strings = {"abcdefgh", "password", "holamundo"})
void faltaMayusculas(String password) {
    checker.setCheckUppers(true);

    assertEquals(
            PasswordChecker.ERR_UPPERS,
            checker.check(password, password)
    );
}

// Comprobar que una contraseña sin minúsculas da error
@ParameterizedTest  robmosroc
@ValueSource(strings = {"ABCDEFGH", "PASSWORD", "HOLAMUNDO"})
void faltaMinusculas(String password) {
    checker.setCheckLowers(true);

    assertEquals(
            PasswordChecker.ERR_LOWERS,
            checker.check(password, password)
    );
}
```

3.5. Coincidencia entre contraseñas

passwordIguales() comprueba que dos argumentos idénticos son aceptados. passwordDiferentes() modifica el segundo argumento añadiendo “hola” y espera ERR_NO_MATCH.

```
// Comprobar que dos contraseñas iguales se aceptan
@ParameterizedTest  robmosroc *
@ValueSource(strings = {"abcdefgh", "password", "HOLAMUNDO"})
void passwordIguales(String password) {
    assertEquals(
            PasswordChecker.PASSWORD_OK,
            checker.check(password, password)
    );
}

// Comprobar que dos contraseñas diferentes dan error
@ParameterizedTest  robmosroc *
@ValueSource(strings = {"password1", "HolaMundo", "ABCDEFGH"})
void passwordDiferentes(String password) {
    assertEquals(
            PasswordChecker.ERR_NO_MATCH,
            checker.check(password,  repeatPassword: password + "hola")
    );
}
```

3.6. Contraseña en blanco

passwordEnBlanco() contempla tres entradas: cadena vacía, un espacio y tres espacios. En los tres casos se espera ERR_BLANK, por lo que se comprueba tanto la cadena vacía como las cadenas formadas solo por espacios.

```
//Comprobar que contraseña vacia da error
@ParameterizedTest  robmosroc
@ValueSource(strings = {"", " ", "   "})
void contrasenaEnBlanco(String password) {
    assertEquals(
            PasswordChecker.ERR_BLANK,
            checker.check(password, password)
    );
}
```

3.7. Validación de la configuración

longitudMinimaNegativa() espera IllegalArgumentException al establecer una longitud mínima negativa. longitudMaximaInferiorAMinima() fija el mínimo en 10 y verifica que establecer el máximo en 9 provoca IllegalArgumentException.

```
// Comprobar que la longitud mínima no puede ser negativa
@Test  robmosroc
void longitudMinimaNegativa() {
    assertThrows(
            IllegalArgumentException.class,
            () -> checker.setMinLength(-1)
    );
}

// Comprobar que la longitud máxima no puede ser inferior a la mínima
@Test  robmosroc
void longitudMaximaInferiorAMinima() {
    checker.setMinLength(10);

    assertThrows(
            IllegalArgumentException.class,
            () -> checker.setMaxLength(9)
    );
}
```

3.8. Getters y valores iniciales

getMinLengthInicial() espera 8; getMaxLengthInicial() espera Integer.MAX_VALUE. checksIniciales() verifica que las opciones de comprobación de dígitos, mayúsculas y minúsculas están desactivadas por defecto.

```
// Comprobar el valor inicial de la longitud mínima
@Test  robmosroc
void getMinLengthInicial() { assertEquals( expected: 8, checker.getMinLength()); }

// Comprobar el valor inicial de la longitud máxima
@Test  robmosroc
void getMaxLengthInicial() { assertEquals(Integer.MAX_VALUE, checker.getMaxLength()); }

// Comprobar los valores iniciales de las comprobaciones
@Test  robmosroc
void checksIniciales() {
    assertFalse(checker.getCheckDigits());
    assertFalse(checker.getCheckUppers());
    assertFalse(checker.getCheckLowers());
}
```

4. Test que se espera que falle y posible corrección

Test afectado: longitudMaximaIncorrecta()

El propio comentario del código indica que el test está bien diseñado, pero falla. Con maxLength=10, las contraseñas de longitud 11 y 23 deberían devolver ERR_LENGTH. Si PasswordChecker las acepta, el comportamiento de la implementación no coincide con el requisito probado.

```
    //comprobar que una longitud mayor de la indicada en maxlength da error
    //El test está bien diseñado pero da error. Habría que comunicarselo a desarrollo
    @ParameterizedTest  robmosroc
    @ValueSource(ints = { 11,23})
    void longitudMaximaIncorrecta(int longitud) {
        checker.setMaxLength(10);

        String password = "A".repeat(longitud);

        assertEquals(
                PasswordChecker.ERR_LENGTH,
                checker.check(password, password)
        );
    }
```

La corrección debe realizarse en PasswordChecker, no debilitando el test: hay que revisar la condición que valida la longitud máxima y asegurar que una contraseña cuya longitud sea mayor que maxLength se rechaza. Conceptualmente, la validación debe detectar password.length() > maxLength y devolver ERR_LENGTH.

Sin el código fuente de PasswordChecker ni la salida de ejecución, no es posible confirmar la línea exacta del defecto ni descartar que exista otra causa. El diagnóstico anterior se basa en el comentario incluido junto al test.

6. Conclusión

Los test cubre nlos principales comportamientos del doc: límites de longitud, presencia de dígitos y tipos de letra, coincidencia, entradas en blanco, configuración y getters. El fallo señalado se concentra en la validación de la longitud máxima. Debe corregirse la implementación para que rechace contraseñas más largas que el límite configurado y, después, ejecutar de nuevo todos los test para comprobar que los demás comportamientos siguen siendo correctos.